

Reliable and Trustworthy AI — Assignment #3

Marabou 를 활용한 MNIST CNN 의 ℓ_∞ 견고성 검증

2026-05-13

1. 모델·데이터셋·검증 쿼리

모델. Assignment #1 에서 학습한 MNIST_CNN 을 그대로 재사용한다. 구조는 Conv(1→32,k=3) → ReLU → MaxPool(2) → Conv(32→64,k=3) → ReLU → MaxPool(2) → Flatten → FC(1600→128) → ReLU → Dropout(eval 시 identity) → FC(128→10) 이다. BatchNorm 이 없고 활성화함수는 ReLU 만 사용해 Marabou ONNX 파서와 호환된다. 체크포인트(mnist_cnn.pth)의 테스트 정확도는 약 **99.1 %**.

데이터. MNIST 테스트셋. 정규화(mean 0.1307, std 0.3081) 입력을 사용한다. 검증 대상으로 두 개의 대비되는 샘플을 선정했다.

- **Sample 0** (digit 7): logit margin = **18.75** — 매우 confident
- **Sample 95** (digit 4): logit margin = **5.30** — 결정경계 근처. 2 위는 digit 8

낮은 margin 후보는 첫 100 개 샘플에 대해 scripts/scan_margins.py 로 스캔하여 골랐다.

검증 쿼리. 각 (sample, ϵ) 에 대해 ℓ_∞ 견고성을 검사한다.

$$\exists x' ? \|x' - x\|_\infty \leq \epsilon \wedge \operatorname{argmax} f(x') \neq \operatorname{argmax} f(x)$$

위 명제는 다음 9 개의 단일 부등식 검증으로 분해된다 (대안 클래스 c' 별):

- 입력 제약: $\max(x_i - \epsilon, X_{\min}) \leq v_i \leq \min(x_i + \epsilon, X_{\max})$ (정규화된 픽셀 유효 범위 안에서 ϵ -ball)
- 출력 제약: $\operatorname{logit}[c_{\text{true}}] - \operatorname{logit}[c'] \leq 0$ (SAT $\Rightarrow c'$ 가 더 높은 logit 을 가지는 입력 존재)

$\epsilon \in \{0.005, 0.01, 0.015, 0.02\}$ 정규화 픽셀 단위 (raw pixel 환산 $\approx 0.0015 \sim 0.0062$).

ONNX export. PyTorch $\rightarrow$ ONNX 시 opset_version=13, dynamo=False (레거시 TorchScript exporter) 를 사용했다. PyTorch 2.6 이상의 기본 dynamo 기반 exporter 는 Shape, Identity 등 maraboupy 가 미지원하는 op 을 삽입한다. PyTorch / onnxruntime / Marabou 세 곳의 출력 일치 차이는 5.96×10^{-7} 으로 충분히 작다.

2. 결과

sample	margin	ϵ (norm)	ϵ (pixel)	status	UNSAT / OOM	total time
0	18.75	0.005	0.0015	UNSAT	9 / 0	46.1 s
0	18.75	0.010	0.0031	UNSAT	9 / 0	46.8 s
0	18.75	0.015	0.0046	UNSAT	9 / 0	46.7 s
0	18.75	0.020	0.0062	UNSAT	9 / 0	46.6 s
95	5.30	0.005	0.0015	OOM*	8 / 1	41.3 s
95	5.30	0.010	0.0031	OOM*	5 / 4	25.5 s
95	5.30	0.015	0.0046	OOM	0 / 9	0.0 s
95	5.30	0.020	0.0062	OOM	0 / 9	0.0 s

* 부분 검증: 8 (또는 5) 개의 대안 클래스에 대해 UNSAT 으로 결론 났으나 나머지가 OOM 으로 미해결.

해석.

Sample 0 은 모든 ϵ 에서 **9 / 9 UNSAT** 으로 Marabou 가 “어떤 ϵ -ball 내의 입력도 7 외의 클래스로 분류될 수 없음” 을 형식적으로 입증했다. Assignment #1 에서 사용했던 FGSM/PGD 같은 attack-only 기법은 “공격을 못 찾았다” 이상은 보장하지 못한다. SMT 기반 검증의 본질적 강점이다.

Sample 95의 결과는 graceful degradation 을 보여준다. $\epsilon = 0.005$ 에서 9 개 비교 중 8 개는 UNSAT 이지만 alt=8 (digit 4 vs digit 8) 만 OOM 으로 미해결이다. ϵ 가 커지면 OOM 영역이 {8} → {8, 9, 6, 0} → 전체로 확장된다.

흥미로운 점은 OOM 순서가 단순히 logit margin 순서를 따르지 않는다는 것이다. sample 95의 logits 에서 4 와의 gap 은 8 (5.3), 9 (7.25), 6 (13.5), 0 (17.2) 순인데, 실제 OOM 순서도 비슷하지만 alt=0 이 alt=6 보다 먼저 OOM 되는 등 단조 정렬이 깨진다. 이는 Marabou 의 search tree 가 단순 margin 보다 **클래스 페어의 결정경계 기하학적 복잡도** 에 더 민감함을 시사한다 — 시각적으로 혼동되기 쉬운 4 ↔ 8 쌍이 가장 먼저 폭발한다.

별도 probe (scripts/probe_eps03.py) 로 sample 0 + $\epsilon=0.03$ + 단일 alt 쿼리만으로도 **RSS ≈ 15 GB / 32 초 후 SIGKILL** 됨을 확인했다. 이는 본 환경(16 GB WSL)의 실질 ϵ 상한을 정한다. counterexample (SAT) 은 도달 가능한 (sample, ϵ) 조합에서는 발견하지 못했다. SAT 를 유도하려면 더 큰 ϵ 또는 더 낮은 margin 의 샘플이 필요한데, 둘 다 즉시 OOM 영역으로 들어간다.

3. Marabou — 강점과 한계 (체험 기반)

강점.

- **형식적 보장.** UNSAT 는 “어떤 perturbation 도 없다” 라는 강한 quantifier-over-input 보증이다.
- **ONNX 직접 입력.** Marabou.read_onnx(...) 한 줄. MarabouUtils.Equation + addInequality API 도 직관적이다.
- **풍부한 resources/.** 36+ ONNX 모델 (ACAS XU, MNIST, CIFAR, layer-zoo 단위 테스트 등), 7 개의 MNIST FCN (.nnet), 64+ 개의 사전 작성된 MNIST robustness property.
- **Per-query timeout 과 verbosity 옵션** 을 통한 runtime 제어 가능.

한계 (실제로 부딪힌 6 건).

1. **Python 버전 제약.** wheel 은 3.8 ~ 3.11 만 지원. Ubuntu 24.04 의 3.12 와 충돌 → 미니콘다로 3.11 환경 분리하는 우회 필요.
2. **PEP 668.** Ubuntu 24.04 의 시스템 pip 는 직접 설치를 차단 (externally-managed-environment). venv / conda 필수.
3. **ONNX 미지원 op.** PyTorch 2.11 의 기본 dynamo exporter 가 Shape, Identity 등을 삽입해 NotImplementedError 발생. dynamo=False 로 회피.
4. **API 혼동.** MarabouCore.Equation (C++ raw 바인딩) vs MarabouUtils.Equation (Python 측 wrapper). 전자를 직접 사용하면 getInputQuery() 단계에서 TypeError. wrapper 를 써야 함.
5. **Disjunction 의 메모리 폭발.** addDisjunctionConstraint([[eq_c1], ..., [eq_c9]]) 는 SIGKILL. 같은 네트워크에서 동일 9 개 제약을 sequential per-class inequality 로 풀면 45 초에 정상 종료. disjunction 분기가 곱셈적으로 누적되는 듯하다.
6. **메모리의 누적성.** 동일 Python 프로세스에서 Marabou.read_onnx + solve 를 반복하면 C++ 측 메모리가 GC 로 회수되지 않고 누적되어 OOM. **subprocess 격리** (verify_query.py) 로 해결. 자식 프로세스 시작 오버헤드 약 3 초/쿼리 (~16%) 를 감수하는 대신 누적 RSS 가 단일 쿼리 피크 (5 GB) 로 묶인다.

Practical takeaway. Marabou 는 search space 가 작은 경우 (margin 충분히 크거나, 모델이 얇거나, ϵ 가 매우 작은 경우) 깔끔한 보증을 제공한다. 반대로 결정경계 근처 입력 + Conv+MaxPool 조합처럼 search 가 폭발하기 쉬운 시나리오에서는 빠르게 메모리 한계에 부딪힌다. 즉, 검증 예산은 모델 단위가 아니라 (**샘플, ϵ**) 단위로 잡아야 한다. 이는 실제 안전 critical 시스템에 적용할 때 “어떤 입력 영역까지 검증 가능한가” 를 사전에 모델·하드웨어 페어 단위로 측정·문서화해야 함을 의미한다.